

ENTERPRISE AI SECURITY GATEWAY

Sentinell.AI

Full Architecture, Working Flow, Features, Tech Stack, Security and Implementation Map

Implementation Snapshot

Django + PostgreSQL backend, Gemini Chrome extension, seven-agent security orchestration, local semantic NLP with optional Groq escalation, response monitoring, file scanning, OpenAI-compatible LLM gateway, Ollama integration, OIDC readiness and an ML-KEM-1024 hybrid token vault.

Document purpose	Panel explanation, technical review and implementation reference
Project state	Working evaluation build with production-oriented deployment controls
Implementation audited	9 June 2026
Code footprint	126 source/config files, approximately 15,788 lines, 176 test methods

Prepared from the current repository implementation, not from planned claims alone.

Contents and Reading Guide

This report follows the system from business problem to runtime execution. Sections 1-6 explain the architecture and working flow. Sections 7-14 describe implementation details. Sections 15-19 cover deployment, evidence, limitations, file ownership and the panel demonstration.

- 1. Executive summary and problem definition
- 2. Current implementation snapshot
- 3. Full system architecture
- 4. End-to-end working flows
- 5. Agentic security and semantic NLP
- 6. Decisions, policy and risk scoring
- 7. Tokenization, encryption and the post-quantum vault
- 8. PostgreSQL database design
- 9. Authentication, RBAC and Zero Trust identity
- 10. Chrome extension and Gemini protection
- 11. Universal LLM middleware gateway and Ollama
- 12. File and attachment scanning
- 13. Web dashboard, audit UI and policy controls
- 14. Complete API catalog
- 15. Full technology stack
- 16. Deployment and operations
- 17. Testing and implementation evidence
- 18. Source-code ownership map
- 19. Benefits, limitations, production roadmap and panel demo

Status language used in this report

Implemented means working code exists now. Configuration-ready means the code path exists but needs external credentials or infrastructure. Production hardening means the current implementation works for evaluation but needs stronger operations at scale.

1. Executive Summary

Sentinell.AI is a Zero Trust security gateway for generative AI usage. It prevents sensitive information from leaving an organization through prompts, files or generated responses. The system evaluates every protected interaction, masks personal data when the workflow can continue safely, and blocks credentials, source code, adversarial intent and other high-risk content before it reaches an LLM.

The project supports two practical integration modes. The Gemini Chrome extension protects a public LLM website at the browser layer. The provider-neutral gateway protects internal applications and private LLMs by placing Sentinell between the company application and a provider such as Ollama, Groq, OpenAI, xAI or another OpenAI-compatible service.

Core security invariant

A provider is called only after outbound content has been inspected. The provider receives the sanitized prompt, not the original sensitive value. Returned content is inspected again before it reaches the user or application.

1.1 Problem addressed

- Shadow AI: employees use public LLMs without centralized organizational visibility.
- Data leakage: PII, credentials, internal code, SQL, environment variables and confidential documents can be pasted into prompts.
- Context gap: static keyword lists miss paraphrased exfiltration intent and unknown identifier formats.
- Response risk: an LLM can generate or repeat secrets, debug dumps or personal data even when the original prompt looked harmless.
- Governance gap: companies need identity-aware policy, audit evidence and an integration layer that does not depend on a single LLM vendor.

2. Current Implementation Snapshot

Area	Current state	Implementation
Backend	Implemented	Python 3.12 and Django with server-rendered UI plus JSON APIs.
Database	Implemented	PostgreSQL configured as the default database; local demo uses port 55432.
Browser protection	Implemented	Chrome Manifest V3 extension for Gemini prompt, response and attachment monitoring.
Agentic analysis	Implemented	Seven specialized agents coordinated by SecurityOrchestrator.
Semantic NLP	Implemented	Local feature-hashing similarity model plus local heuristics; optional Groq/Grok/OpenAI escalation.
Post-quantum vault	Implemented	ML-KEM-1024 + HKDF-SHA256 + AES-256-GCM envelope for TokenMap values.
Response firewall	Implemented	ALLOW, REDACT or BLOCK on generated output.
File scanning	Implemented	PDF, DOCX, text/config/source formats and optional OCR images.
Private LLM gateway	Implemented	OpenAI-compatible middleware integrated with local Ollama llama3.2.
Enterprise SSO	Configuration-ready	OIDC authorization-code flow, JWT verification, role/group mapping and identity context.
Policy authoring	Implemented via admin	PolicyRule model and Django admin; custom UI currently displays rules rather than editing them.

2.1 Current configured demo profile

Setting	Audited value
Database engine	django.db.backends.postgresql
Database	sentinell_ai_db on 127.0.0.1:55432
Approved-response provider	openai_compatible
Private model	Ollama llama3.2
Local semantic classifier	Enabled
Semantic escalation provider	Groq configured
Vault provider	ml-kem-1024-aesgcm-v1
OIDC SSO	Code implemented; disabled in the local demo

3. Full System Architecture

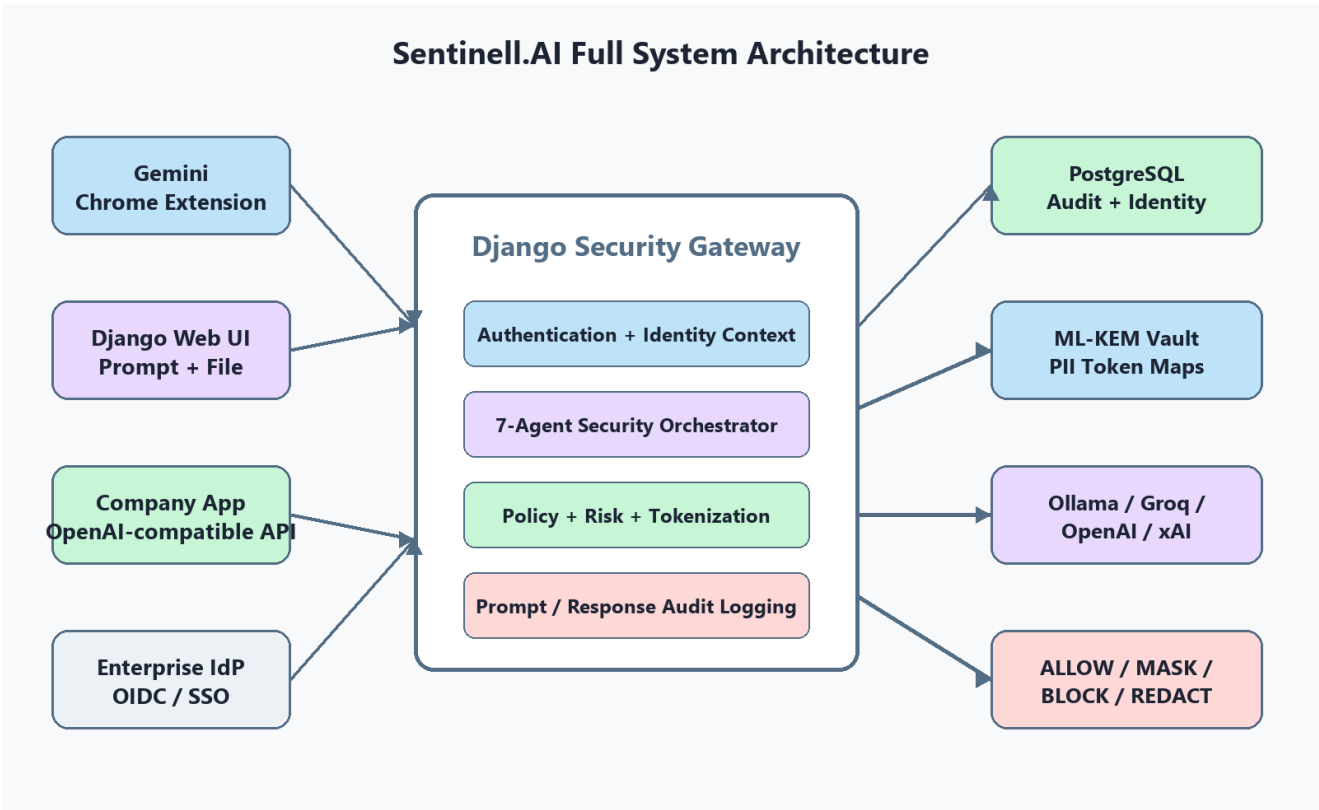


Figure 1. Input channels, central security gateway, protected storage and LLM providers.

3.1 Architectural layers

Layer	Responsibility	Key files
Interaction	Gemini extension, Django pages and company application requests.	chrome_extension/*; templates/*
Authentication	Local sessions, extension bearer tokens and OIDC identity provisioning.	users/views.py; users/api_views.py; users/oidc.py
Ingress APIs	Prompt, response, file and OpenAI-compatible gateway endpoints.	firewall/extension_api.py; firewall/gateway_api.py
Security orchestration	Runs deterministic, contextual and semantic agents and combines evidence.	firewall/agents/orchestrator.py
Policy and risk	Role policy, additive organization rules, decision priority and severity scoring.	policy_engine.py; policy_rules.py; risk_scorer.py
Protection	Masking, encrypted logs and ML-KEM token vault storage.	tokenization.py; encryption.py; vault.py
Provider routing	Calls mock, Groq, OpenAI, xAI or generic OpenAI-compatible providers.	llm_gateway.py
Persistence	Stores users, tokens, prompt/response logs, token maps and policy rules.	models.py; PostgreSQL
Operations	Health checks, migrations, static assets, Gunicorn and Docker orchestration.	health.py; entrypoint.sh; Dockerfile; compose files

4. End-to-End Working Flows

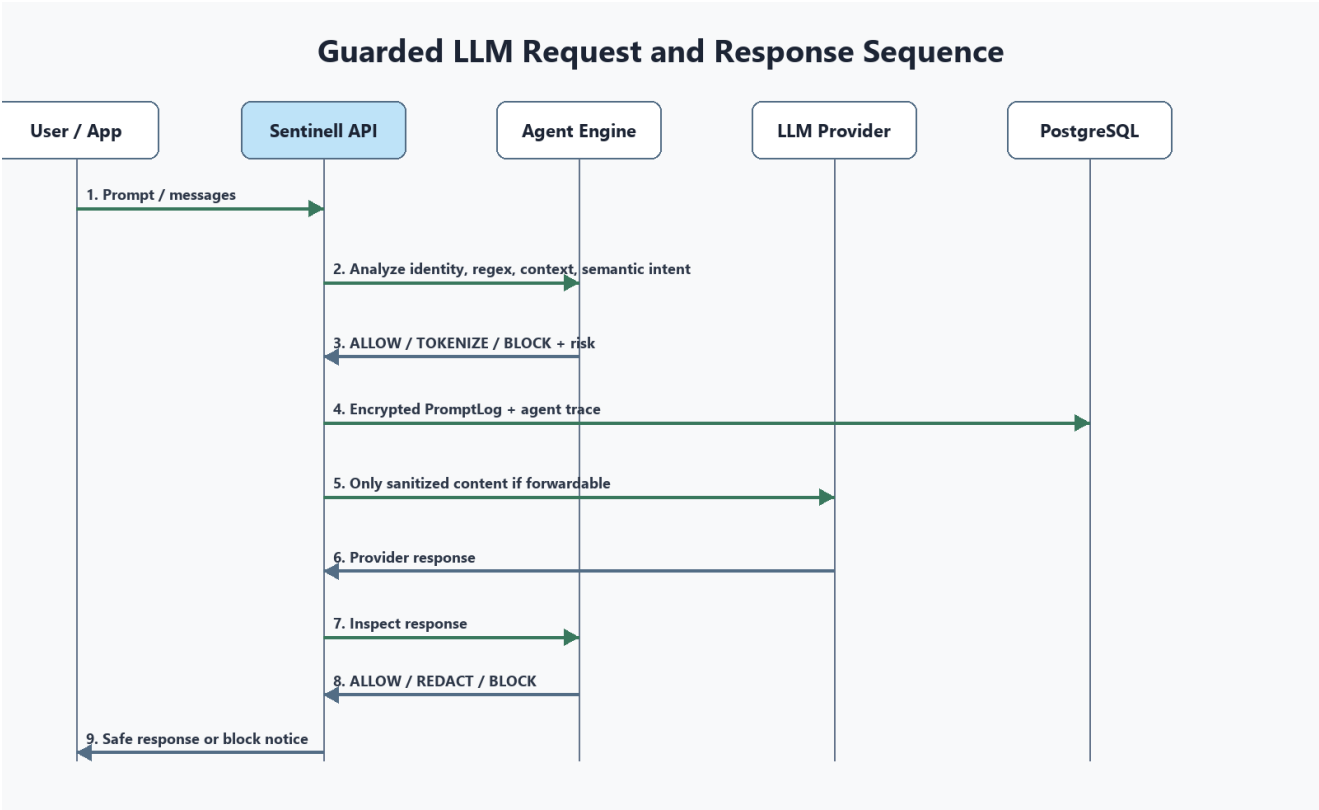


Figure 2. A prompt is inspected before the provider call and the response is inspected after it.

4.1 Gemini prompt protection

1. The content script captures a click on Gemini's send button or an Enter-key submission.
2. It reads the active Gemini editor and prevents the browser event from reaching Gemini.
3. The extension background worker sends the prompt to POST /firewall/check with a bearer token.
4. Django authenticates the token, attaches the user's role and identity context, then runs SecurityOrchestrator.
5. ALLOW triggers one temporary approved submission. TOKENIZE opens a warning panel with Copy masked prompt and Replace prompt. BLOCK suppresses submission.
6. The backend writes an encrypted PromptLog and stores masked-value mappings in TokenMap.

4.2 Gemini response monitoring

1. A MutationObserver watches Gemini response DOM changes and schedules scans after the output stabilizes.
2. Candidate response elements are filtered to avoid user prompts, editor text and Sentinell's own warning panel.
3. The response is sent to POST /firewall/check-response.
4. ALLOW leaves the answer visible. REDACT offers the protected version. BLOCK applies the CSS class sentinell-response-hidden and shows a safe notice.
5. The original output is encrypted in ResponseLog together with the decision, source, risk and agent trace.

4.3 Private/company LLM gateway

```
Company application
-> Authorization: Bearer <Sentinell token>
-> POST /gateway/v1/chat/completions
-> inspect every system/developer/user/tool message
-> mask or block
-> call Ollama or another configured provider
-> inspect provider response
-> return safe OpenAI-compatible response + Sentinell metadata
```

For the current Ollama demonstration, Sentinell calls <http://localhost:11434/v1/chat/completions> using the generic openai_compatible adapter and model llama3.2. Local Ollama does not require a real provider key; the configured placeholder satisfies the common Authorization header shape.

5. Agentic Security and Semantic NLP

The term agentic refers to specialized components with separate responsibilities and a shared orchestration contract. The system does not let an LLM autonomously decide everything. Deterministic controls remain authoritative, while semantic classification adds context for intent that regex cannot express reliably.

Agent	Role	Typical evidence/action
PIIAgent	Known personal identifiers.	Email, phone, Aadhaar, passport, IP and MAC; TOKENIZE.
ContextualPIIAgent	Unknown-format values labeled by context.	Employee code, patient ID, DOB, account reference; TOKENIZE.
SecretsAgent	Credentials and protected technical material.	API keys, passwords, cloud keys, private keys; BLOCK.
PromptInjectionAgent	Jailbreak and instruction manipulation.	Role override, encoded payload, social engineering; BLOCK.
SemanticSecurityAgent	Intent beyond literal patterns.	Exfiltration, prompt extraction, policy bypass, privilege escalation.
PolicyAgent	Role-aware Zero Trust decision.	ADMIN, EMPLOYEE and INTERN rule explanation.

Agent	Role	Typical evidence/action
RiskAgent	Severity and defense-in-depth.	0-100 score, level and should_block.

5.1 Detection foundation

scanner.py defines 25 detection categories: credit_card, financial_account, password, encryption_key, private_key, ssn, aadhaar, email, phone, mac_address, ip_address, passport, phone_words, api_key, cloud_key, source_code, sql_query, documentation, debug_env_dump, credential_request, adversarial_injection, encoded_payload, secret_token, embedded_secret_key and social_engineering_injection.

Regex is intentionally not the only layer. Contextual expressions detect short or irregular phone numbers when the surrounding language labels them as personal contact data, and detect previously unknown identifiers when phrases such as employee code, patient number, customer ID or date of birth establish sensitive context.

5.2 Local semantic service

- Known detected values are replaced with [REDACTED_TYPE] before any optional external semantic request.
- High-precision local heuristics catch environment exfiltration, credential requests, privilege escalation and hidden prompt extraction.
- The local model normalizes semantic aliases and embeds words, bigrams, trigrams and character features into a deterministic 4,096-dimensional feature-hashed vector.
- Cosine similarity compares the prompt with risky intent prototypes and safe prototypes.
- A block threshold, escalation threshold and minimum similarity margin determine BLOCK, local ALLOW or second-opinion escalation.
- The agent trace records model name, intent, risky similarity, safe similarity, margin, matched prototype and escalation status.

Important technical distinction

The local semantic component is an NLP similarity classifier, not a trained neural transformer model. It is private, deterministic and lightweight. Groq, Grok or OpenAI can provide an optional LLM second opinion for uncertain cases.

6. Decisions, Policy and Risk Scoring

Direction	Decision	Runtime effect
Prompt	ALLOW	Content passes to Gemini/provider unchanged unless tokenization targets also exist in an explicitly allowed admin workflow.
Prompt	TOKENIZE	Sensitive spans are replaced; only the masked prompt is forwardable.
Prompt	BLOCK	No provider call or website submission occurs.
Response	ALLOW	The response is displayed or returned normally.
Response	REDACT	PII is replaced and a protected response is returned.
Response	BLOCK	Unsafe output is hidden and replaced with a safe notice.

6.1 Built-in role policy

Role	PII	Technical content	Universal high-risk data
ADMIN	Masked	SQL and documentation may be allowed; source code remains universally blocked.	Blocked
EMPLOYEE	Masked	SQL blocked; unknown sensitive types fail safe.	Blocked
INTERN	Masked	SQL and documentation blocked; document upload also restricted in web/API paths.	Blocked

6.2 Additive PolicyRule engine

Administrators can define enabled rules with priority, action, direction, roles, excluded roles, detection types, keywords, source substring and minimum risk score. Rules are additive: they can make a decision stricter but cannot weaken universal blocks.

6.3 Risk calculation

Case	Score behavior	Result
No detection	0	LOW; allow
PII only	Fixed 40	MODERATE; tokenize, never block solely for PII
High-risk only	60-75 based on severity	HIGH/SEVERE; block
PII + high-risk	80-95 based on severity	SEVERE/CRITICAL; block
Semantic attack	Minimum elevated score 85	CRITICAL; block

7. Tokenization, Encryption and Post-Quantum Vault

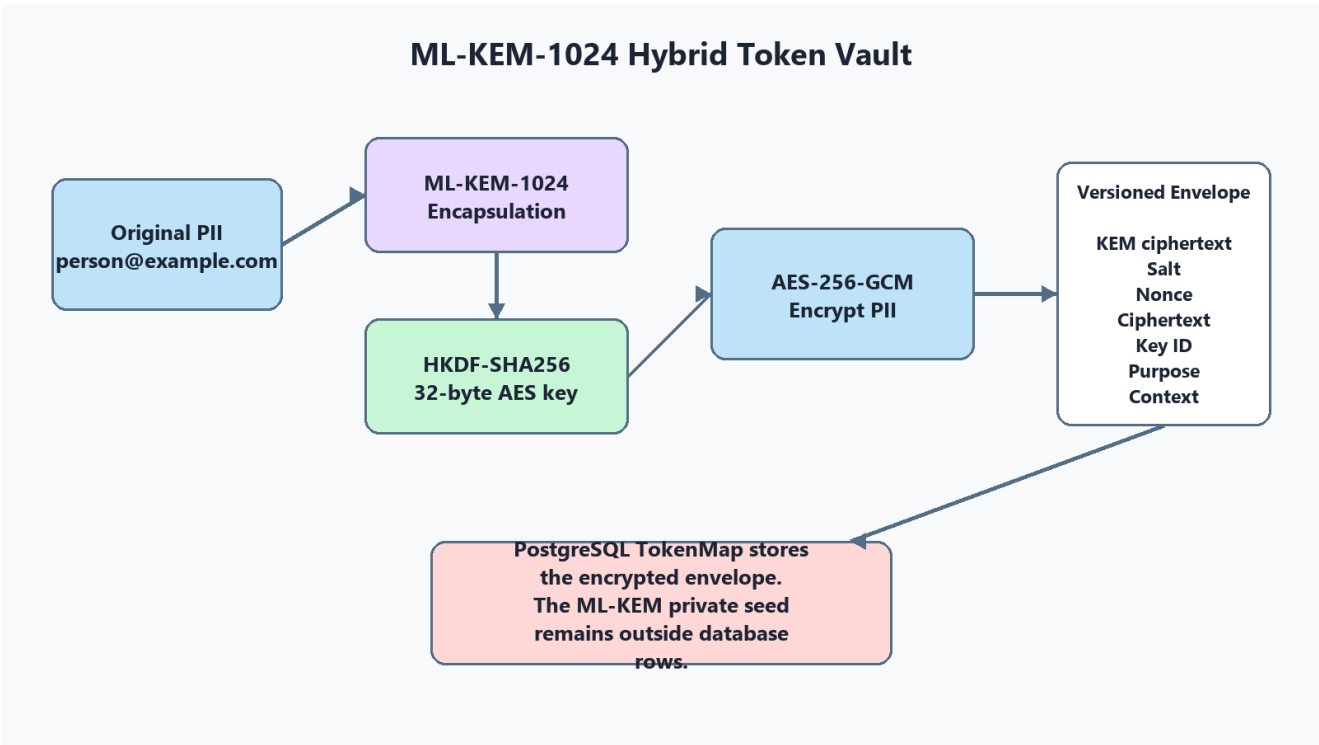


Figure 3. ML-KEM establishes the shared secret; AES-GCM encrypts the actual PII value.

7.1 Masking behavior

Data type	Example protected output
Email	person@example.com -> p***@***.com
Phone	9876543210 -> *****3210
Aadhaar	2345 6789 0123 -> XXXX-XXXX-0123
Credit card	4111 1111 1111 1111 -> **** * 1111
SSN	123-45-6789 -> ***-**-6789
IP address	192.168.1.100 -> ***,***,***,100
Unknown contextual ID	EMP-91-KL7 -> type-aware default mask

7.2 Encryption responsibilities

Protected item	Mechanism	Storage
Original prompt	Fernet authenticated symmetric encryption	PromptLog.original_prompt
Original model response	Fernet authenticated symmetric encryption	ResponseLog.original_response
Original value behind a mask	ML-KEM-1024 KEM + HKDF-SHA256 + AES-256-GCM	TokenMap.encrypted_value envelope
Extension/gateway token	Random token shown once; SHA-256 hash stored	ExtensionToken.token_hash
User password	Django password hasher and validators	CustomUser inherited password field

7.3 Where the ML-KEM private seed is stored

In the current implementation, MLKEM1024_PRIVATE_KEY and MLKEM1024_PUBLIC_KEY are loaded from environment configuration. The private seed is not stored in TokenMap or PromptLog rows. vault.py reconstructs an MLKEM1024PrivateKey only when opening a sealed value. Production should move this secret from a plain environment file to a managed secret manager, HSM or KMS-backed injection mechanism and add key rotation and access auditing.

8. PostgreSQL Database Design

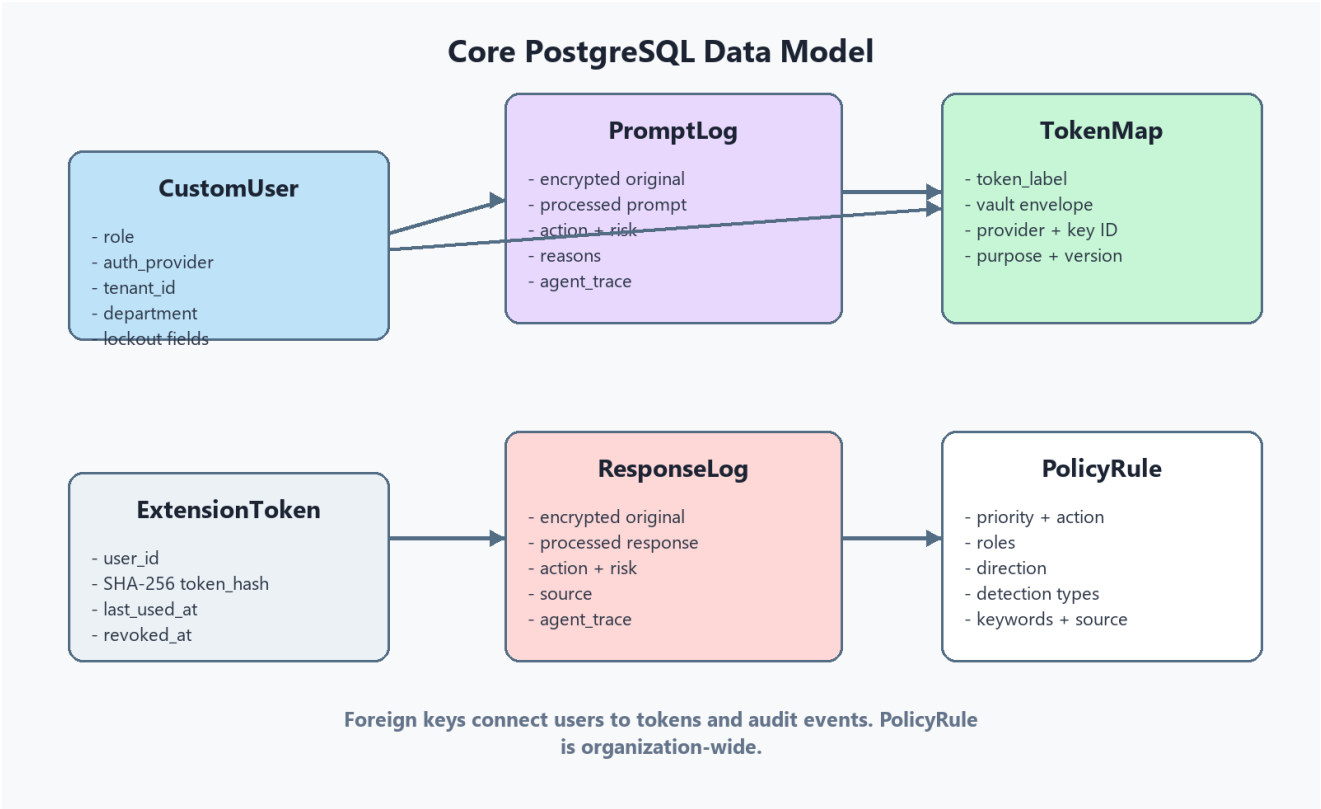


Figure 4. Main application models and their storage responsibilities.

Model	Important fields	Purpose
CustomUser	role, provider, subject, tenant, department, claims, lockout	Identity, RBAC and Zero Trust context.
ExtensionToken	user, token_hash, created, last_used, revoked	Bearer authentication for extension and gateway clients.
PromptLog	encrypted original, processed prompt, action, risk, reasons, trace	Outbound audit event.
ResponseLog	encrypted original, processed response, action, risk, source, trace	Inbound model-output audit event.
TokenMap	label, vault envelope, provider, key ID, purpose, version	Protected original-to-mask mapping.
PolicyRule	priority, direction, roles, types, keywords, risk threshold	Organization-specific additive controls.

Why PostgreSQL

The project uses relational integrity for users and audit events, JSON fields for flexible detections and agent traces, transactions and mature indexing/backup capabilities. SQLite is not the configured application database.

9. Authentication, RBAC and Zero Trust Identity

- Local login uses email lookup plus Django authentication and CSRF-protected forms.
- Five failed logins lock the account for 30 minutes.
- RateLimitMiddleware restricts login/signup POST requests to 10 per minute per observed IP.

- Extension login issues a random URL-safe bearer token; only its SHA-256 hash is stored.
- Every extension and gateway request resolves the token back to a user and updates last_used_at.
- Roles are ADMIN, EMPLOYEE and INTERN and are attached to the orchestrator decision.
- Agent traces include auth provider, external subject, tenant, department and role.

9.1 OIDC enterprise SSO

The authorization-code flow generates state and nonce, exchanges the code at the configured token endpoint, verifies the ID token with JWKS and allowed algorithms, maps groups to roles, and creates or updates a CustomUser. It can integrate with Microsoft Entra ID, Okta, Auth0, Keycloak or another OIDC-compatible provider once deployment credentials are configured.

Current scale limitation

The login rate limiter uses process-local memory. Multi-worker production deployment should replace this with Redis or another shared store.

10. Chrome Extension and Gemini Protection

Extension file	Responsibility
manifest.json	Manifest V3 permissions, Gemini match pattern, popup and background service worker.
content.js	Intercepts send, locates Gemini editor/buttons, replaces prompts, observes responses and intercepts file selection.
background.js	Stores configuration, authenticates, and performs backend fetch calls.
popup.html / popup.css / popup.js	Login and controls for prompt protection, response monitoring, attachment scan and safe preview.
content.css	Warning panel styling and the response-hidden blur state.

10.1 Why it behaves differently from the Ollama desktop app

Gemini runs in a browser page where a Chrome extension can inspect and modify DOM elements. The native Ollama desktop application is not a Chrome page, so content.js cannot replace its message bubble. Protection is still possible through Ollama's API: a company application must call Sentinell first, and Sentinell forwards only the masked prompt to Ollama.

10.2 Current extension scope

The manifest currently matches `https://gemini.google.com/*`. ChatGPT, Claude and other sites are not active in this build. Adding them requires site-specific editor, send-button and response selectors plus regression tests because these websites change frequently.

11. Universal LLM Middleware Gateway and Ollama

The gateway is the enterprise architecture. Company applications do not call an LLM provider directly. They authenticate to Sentinell, which inspects all outbound messages, routes only approved content, scans the provider response and returns a protected result.

Provider ID	Purpose	Credential behavior
mock	Offline deterministic demonstration provider.	No API key.
groq	Groq OpenAI-compatible chat completions.	Real GROQ_API_KEY required.

Provider ID	Purpose	Credential behavior
openai	OpenAI chat completions adapter.	Real OPENAI_API_KEY required.
xai	xAI/Grok chat completions adapter.	Real XAI_API_KEY required.
openai_compatible	Ollama, vLLM or a hosted private endpoint.	Local Ollama can use a placeholder; hosted services normally require a real service token.

11.1 Gateway authentication separation

- Client-to-Sentinell token: proves which user/application is making the gateway request.
- Sentinell-to-provider key: held only by the Django backend and used to call the configured LLM.
- The browser or company frontend never receives the provider key.
- The current gateway reuses ExtensionToken. A production enhancement is a separate application-token model with scopes, expiry, rotation and admin revocation UI.

11.2 Gateway demo page

The authenticated `/gateway-demo/` page displays the original prompt, Sentinell decision, exact prompt forwarded to Ollama, provider/model/fallback state and the post-firewall response. It is intended to make the middleware architecture visible to evaluators.

12. File and Attachment Scanning

Category	Supported formats / behavior
Documents	PDF with pdfplumber; DOCX paragraphs and table cells with python-docx.
Structured text	CSV, JSON, JSONL, logs, SQL, Markdown, XML, YAML and config files.
Source text	Python, JavaScript/TypeScript, Java, C/C++, C#, Go, Rust, PHP and Ruby.
Images	PNG, JPG, BMP, TIFF and WebP through Pillow + optional Tesseract OCR.
Limits	Default 5 MB upload and 100,000 extracted characters.
Extension behavior	Only ALLOW re-releases the original file to Gemini. TOKENIZE/BLOCK stops upload because the original file still contains sensitive data.

The extraction layer does not decide safety. It produces text, then reuses the same SecurityOrchestrator, policy, risk, token vault and audit pipeline as typed prompts. This keeps behavior consistent across text and document channels.

13. Web Dashboard, Audit UI and Policy Controls

Page	Working capability
/dashboard/	Health score, decisions, intervention rate, risk, semantic mode, vault status and recent events.
/prompt/	Authenticated prompt/file analysis with full security report and provider response.
/gateway-demo/	Visible company-app -> Sentinell -> Ollama middleware demonstration.
/logs/	Admin audit list and deletion of old prompt logs.
/logs/<id>/	Masked sensitive fields with a server-enforced 10-second reveal window.
/policy-rules/	Admin view of enabled organization rules and summary.
/admin/	Django administration for users, policies and protected audit models.

14. Complete API Catalog

Method and endpoint	Authentication	Purpose
GET /healthz/	None	Application and PostgreSQL health.
POST /extension/login/	Email/password	Issue extension bearer token.
POST /firewall/check	Bearer token	Full prompt firewall.
POST /firewall/check-response	Bearer token	Full response firewall.
POST /firewall/check-file	Bearer token	Extension file scanner.
POST /gateway/chat	Bearer token	Native provider-neutral gateway API.
POST /gateway/v1/chat/completions	Bearer token	OpenAI-compatible protected completion.
GET /gateway/providers	Bearer token	Configured provider catalog and availability.
POST /gateway/files/scan	Bearer token	Company application file pre-scan.
POST /analyze	None	Legacy/basic text SAFE or BLOCK detector.
POST /analyze-file	Session-aware restriction	Legacy/basic file SAFE or BLOCK detector.
GET /oidc/login/	None	Start enterprise OIDC flow.
GET /oidc/callback/	OIDC state/code	Verify identity and provision user.

API boundary note

Production integrations should use the authenticated /firewall/* and /gateway/* endpoints. The simple /analyze endpoints are compatibility utilities and do not provide the full audited tokenized gateway contract.

15. Full Technology Stack

Layer	Technology	Use in Sentinell.AI
Language/runtime	Python 3.12	Backend, agents, extraction, encryption and deployment commands.
Web framework	Django 4.2+	Routing, ORM, sessions, auth, templates, admin and APIs.
Database	PostgreSQL 18	Persistent identity, audit, policies and vault envelopes.
DB driver	psycopg2-binary	Django PostgreSQL connectivity.
Browser	Chrome Manifest V3	Gemini interception, popup and service worker.
Frontend	Django Templates, HTML, CSS, JavaScript, Bootstrap 5, Bootstrap Icons	Operational dashboard and security reports.
Crypto	cryptography 48+	Fernet, ML-KEM-1024, AES-GCM, HKDF and SHA-256.
NLP	Local feature hashing + cosine similarity	Offline semantic risk classification.
Optional semantic LLM	Groq, xAI/Grok or OpenAI	Second opinion for uncertain local scores.
Private model demo	Ollama llama3.2	OpenAI-compatible company LLM backend.
Documents	pdfplumber, python-docx	PDF and DOCX extraction.
OCR/images	Pillow, pytesseract, Tesseract	Optional image text extraction.
SSO	PyJWT + OIDC/JWKS	Enterprise identity validation and provisioning.
Production server	Gunicorn	Multi-worker WSGI runtime.
Static assets	WhiteNoise	Compressed production static serving.
Containers	Docker, Docker Compose	Local and production service orchestration.

16. Deployment and Operations

- Dockerfile uses python:3.12-slim, installs PostgreSQL runtime libraries, Tesseract and Poppler, creates a non-root sentinell user and exposes port 8000.
- docker-compose.yml provides a local PostgreSQL 18 service, persistent volume, health checks and web service startup.
- compose.production.yml isolates PostgreSQL on an internal backend network and exposes Django through a separate edge network.
- entrypoint.sh waits for PostgreSQL, validates production settings, checks migration drift, applies migrations, collects static files and starts Gunicorn.
- validate_deployment rejects development secrets, debug mode, wildcard CORS/hosts, invalid Fernet keys, missing ML-KEM keys and incomplete enabled SSO configuration.
- /healthz/ performs a live SELECT 1 database check and returns HTTP 503 when PostgreSQL is unavailable.

16.1 Production security settings

Control	Implementation
Cookies	HttpOnly, SameSite=Lax, secure flags controlled by production environment.
Transport	SSL redirect, proxy header and HSTS controls available.
Clickjacking	X-Frame-Options DENY.
MIME/referrer	nosniff and strict-origin-when-cross-origin headers.
CORS	Explicit origins expected in production; wildcard rejected by validation.
Secrets	Environment-driven configuration; production recommendation is managed secret storage.

17. Testing and Implementation Evidence

Metric	Current repository result
Source/config files counted	126
Approximate source/config lines	15,788
Python files counted	101
Test methods discovered	176
Scanner detection categories	25
Specialized agents	7
Application routes	22 direct routes plus included URL modules

Tests cover detection, role policy, prompt actions, contextual phone/PII cases, dashboard behavior, audit deletion, file extraction, extension endpoints, response redaction/blocking, ML-KEM vault round trips, gateway routing, OpenAI-compatible responses, OIDC behavior and deployment controls. The gateway masking test specifically asserts that person@example.com is absent from the message passed to the provider.

17.1 Demonstrated Ollama result

```
Status: OK
Prompt action: TOKENIZE
Processed prompt: My email is p***@***.com. Draft a support reply.
Provider: openai_compatible
Model: llama3.2
Fallback: False
Response action: ALLOW
```

18. Source-Code Ownership Map

Capability	Primary implementation files
Project settings and routing	sentinell_ai/settings.py; sentinell_ai/urls.py; sentinell_ai/health.py
Authentication and identity	users/models.py; users/views.py; users/api_views.py; users/oidc.py; users/middleware.py
Web application pages	firewall/views.py; templates/firewall/*; templates/base.html; static/css/style.css
Detection	firewall/scanner.py; firewall/detector.py
Agent orchestration	firewall/agents/orchestrator.py; firewall/agents/*.py
Semantic NLP	firewall/agents/semantic_agent.py; firewall/agents/semantic_embeddings.py
Policy and risk	firewall/policy_engine.py; firewall/policy_rules.py; firewall/risk_scorer.py
Masking and encryption	firewall/tokenization.py; firewall/encryption.py; firewall/vault.py
Extension APIs	firewall/extension_api.py; users/api_views.py
Gateway and providers	firewall/gateway_api.py; firewall/llm_gateway.py
File scanning	firewall/document_extractor.py; firewall/file_scanning.py
Persistence/admin	firewall/models.py; firewall/admin.py; users/admin.py; migrations/*
Chrome extension	chrome_extension/manifest.json; content.js; background.js; popup.*; content.css
Deployment	Dockerfile; docker-compose.yml; compose.production.yml; entrypoint.sh; scripts/*.ps1
Verification	firewall/tests.py; firewall/tests_api.py; users/tests.py; test_dataset/*

19. Organizational Benefits and Use Cases

Company use case	How Sentinell.AI helps
Employees using public AI	Gemini extension prevents accidental PII and secret submission.
Internal support/CRM copilot	Gateway masks customer identifiers before provider calls.
Private engineering assistant	Ollama remains inside the company while Sentinell enforces policy and audit.
Healthcare/finance workflows	Contextual identifiers and PII masks reduce exposure; policies can be made stricter.
Security and compliance teams	Encrypted evidence, risk scores, reasons and agent traces support investigations.
Multi-provider AI strategy	OpenAI-compatible routing avoids tying governance to one model vendor.

19.1 What is unique compared with static filters

- It protects both outbound prompts and inbound responses.
- It preserves productivity by masking useful prompts instead of blocking every sensitive workflow.
- It combines known patterns, contextual unknown identifiers and semantic intent similarity.
- It records explainable evidence from multiple specialized agents.
- It protects token mappings with a real ML-KEM-1024 hybrid envelope.
- It operates both at a browser website layer and as provider-neutral middleware.
- It demonstrates a private company LLM through Ollama without exposing the original PII to the model.

19.2 Current limitations and honest panel answers

Limitation	Correct explanation / next hardening step
Native Ollama desktop prompt is not replaced	The desktop UI bypasses the gateway. Company applications must call Sentinel's API; use /gateway-demo/ to visualize the protected flow.
Extension supports Gemini only	Other websites require maintained site adapters and selector tests.
Local semantic model is not a trained transformer	It is deterministic NLP feature hashing; add a trained local NER/classifier service for stronger ML coverage.
ML-KEM seed is environment-configured	Move it to HSM/KMS/secret manager, add rotation and split access.
Rate limiting is in memory	Use Redis for shared limits and distributed workers.
Gateway token model is shared with extension	Add scoped service accounts, expiry, rotation, per-app policy and revocation UI.
No multi-tenant isolation model yet	Add Organization/Tenant models, tenant-scoped queries and tenant-specific key/policy boundaries.
DOM integration can break when Gemini changes	Maintain selector adapters and automated browser regression tests.

19.3 Recommended production roadmap

1. Introduce Organization, ApplicationCredential and scoped GatewayToken models with rotation and per-app policies.
2. Move secrets and ML-KEM keys into a managed KMS/HSM or secret manager and implement versioned rotation.
3. Add Redis-backed rate limiting, queues and centralized audit/event streaming.
4. Add a trained local NER model and calibrated semantic classifier service while keeping deterministic blocking controls.
5. Expand browser adapters to ChatGPT, Claude, Copilot and other approved sites.
6. Add streaming gateway support and response scanning that preserves token-by-token user experience.
7. Add SIEM/SOC integration, retention policies, metrics, alerting and data residency controls.
8. Add multi-tenant isolation, organization policy authoring UI and provider/model allowlists.

19.4 Recommended final panel demonstration

1. Open the dashboard and explain the two integration modes: Gemini extension and enterprise gateway.
2. Show a safe Gemini prompt and the ALLOW path.
3. Show an email prompt, click Replace prompt and prove Gemini receives the masked value.
4. Show an API key or environment-exfiltration prompt and prove the prompt never submits.
5. Show a generated verbose environment dump and the response blur/block behavior.
6. Upload a safe text file and then a file containing PII or a cloud secret.
7. Open /gateway-demo/, send the same email prompt to Ollama and show Prompt Sent To Ollama contains p***@***.com with fallback No.
8. Open the audit view and show risk, reasons, agent trace and encrypted-at-rest design.
9. Show orchestrator.py, semantic_agent.py, gateway_api.py, llm_gateway.py and vault.py as the five strongest code paths.

Conclusion

Sentinell.AI is no longer only a regex prompt checker. The current implementation is a layered AI security gateway with browser interception, authenticated middleware APIs, contextual and semantic analysis, role-aware policy, response inspection, document scanning, encrypted audit, a post-quantum token vault and private-LLM routing. Its strongest production direction is to become the mandatory control point through which company applications access any approved generative AI provider.

Panel-ready one-line pitch

Sentinell.AI is a Zero Trust firewall for generative AI that masks useful sensitive data, blocks dangerous intent, protects both prompts and responses, and gives companies one governed gateway for public and private LLMs.